

Deriving Interpreters:

From Big-Step SOS to Abstract Machines

Mahmoud M. Hamido

Department of Computer Science
RPTU Kaiserslautern-Landau

Supervised by:

Prof. Dr. Ralf Hinze
Alexander Dinges, M.Sc.

May 20, 2026



Outline

1. Introduction
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work
7. Conclusion

Introduction: I

- > A **specification** describes the behavior of a programming language:
 - > Documents in plain text describing it (e.g, the JLS).
 - > A set of evaluation rules (e.g, the semantics of WASM).
- > An **implementation** ideally adheres to the specification.
- > Deviation from the specification (usually) implies the existence of a bug.

But how would one check that no such mismatch exists?

Introduction: II

Adherence can be verified through either:

- > Manual reviews
- > Testing against a standard test suite, and (optionally) compare results against a *reference implementation*.

Testing does not prove the *absence* of bugs.

Thesis

- Encode the specification (semantics) in a dependently-typed language (Agda).
- Derive an abstract machine (interpreter) from the semantics.
- *Prove* that they are consistent: produce the same results on all programs.¹

¹For programs that terminate.

Outline

1. Introduction
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work
7. Conclusion

Background: Agda

- > Agda is a dependently typed programming language and proof assistant.
- > Key distinction: **parameters** (before the colon) vs **indices** (after the colon).

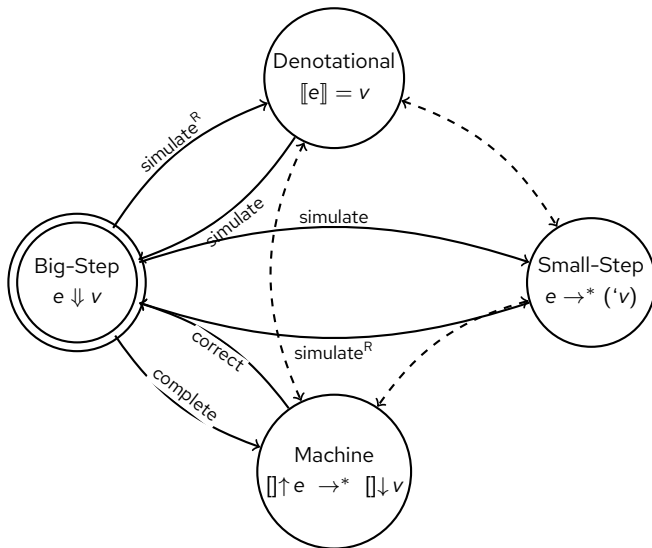
```
data Vec (A : Set) : ℕ → Set where
  []      : Vec A zero
  _::__   : A → Vec A n → Vec A (suc n)
```

- > **Parameters**: fixed across all constructors.
- > **Indices**: each constructor may specialise it to a specific value.

Background: Propositions as Types

Name		Agda Representation	
Truth		Unit type	$\top$
Falsehood		Empty type	$\perp$
Implication		Function type	$P \rightarrow Q$
Negation	$\neg P$	Implication into $\perp$	$P \rightarrow \perp$
Conjunction	$P \wedge Q$	Product	$P \times Q$
Disjunction	$P \vee Q$	Sum	$P \uplus Q$
Existential	$\exists x. P(x)$	Dependent product	$\Sigma (A : \text{Set}) (P : A \rightarrow \text{Set})$
Equality	$a \equiv b$	Indexed	$\text{refl} : x \equiv x$

Background: Semantic Styles



Outline

1. Introduction
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work
7. Conclusion

Arithmetic: Syntax and Intrinsic Typing

```
data Type : Set where
  Nat  : Type
  Bool : Type
```

```
data ⊢_ : Type → Set where
  \_      : ⊢ T → ⊢ T
  _⊞_     : ⊢ Nat → ⊢ Nat → ⊢ Nat
  test    : ⊢ Nat → ⊢ Bool
  if_then_else_ : ⊢ Bool → ⊢ T → ⊢ T → ⊢ T
```

- > The quote constructor `\` embeds values (naturals and booleans) into syntax.
- > Intrinsically typed: only well-typed terms can be constructed.

Arithmetic: Big-Step Semantics

```
data _⇓_ : ⊢ T → ⇨ T → Set where
  \_      : (v : ⇨ T) → (\ v) ⇓ v
  _+_     : e1 ⇓ n1 → e2 ⇓ n2 → (e1 + e2) ⇓ (n1 + n2)
  test    : e ⇓ n → (test e) ⇓ (test' n)
  if-then : e ⇓ true  → e1 ⇓ v → (if e then e1 else e2) ⇓ v
  if-else : e ⇓ false → e2 ⇓ v → (if e then e1 else e2) ⇓ v
```

- > A derivation tree of $e \Downarrow v$ is a proof that e evaluates to v .
- > Properties: **determinism** and **totality**.

Arithmetic: Determinism

deterministic : $e \Downarrow v_1 \rightarrow e \Downarrow v_2 \rightarrow v_1 \equiv v_2$

- > Proof by simultaneous induction on both derivations.
- > Most cases follow immediately: matching constructors imply matching sub-derivations.

Arithmetic: Totality

$\text{total} : \forall (e : \vdash T) \rightarrow \exists (\lambda v \rightarrow e \Downarrow v)$

- > Proof by structural induction on e .
- > Produce the value and the derivation tree from recursive calls onto the subexpressions

Arithmetic: Abstract Machine

```
data Frame : Type → Type → Set where
  _else_    : ⊢ T → ⊢ T → Frame T      Bool
  _⊞₀_      : Hole → ⊢ Nat → Frame Nat Nat
  _⊞₁_      : ⊢ Nat → Hole → Frame Nat Nat
  test      : Hole → Frame Bool        Nat

data Stack (Answer : Type) : Type → Set where
  []        : Stack Answer Answer
  _∷_       : Stack Answer T₁ → Frame T₁ T₂ → Stack Answer T₂
```

- > $\text{Frame } T_1 \ T_2$: indexed by the **result** type of the term, and the expected type of the **hole**.
- > $\text{Stack Answer } T$: a sequence of frames leading from the current focus type T back to the final answer type.

Arithmetic: Abstract Machine

```
data State (Answer : Type) : Set where
```

```
  ⊢_↑_ : Stack Answer T → ⊢ T → State Answer
```

```
  ⊢_↓_ : Stack Answer T → ⊢ T → State Answer
```

```
data _→_ : State Answer → State Answer → Set where
```

```
  step-` : ⊢ stack ↑ ` v → ⊢ stack ↓ v
```

```
  step-⊞1 : ⊢ stack ↑ (e1 ⊞ e2) → ⊢ (stack :: ◯ ⊞0 e2) ↑ e1
```

```
  step-⊞2 : ⊢ (stack :: ◯ ⊞0 e2) ↓ v1 → ⊢ (stack :: v1 ⊞1 ◯) ↑ e2
```

```
  step-⊞3 : ⊢ (stack :: v1 ⊞1 ◯) ↓ v2 → ⊢ stack ↓ (v1 + v2)
```

```
  ...
```

- > ↑: evaluate an expression.
- > ↓: return a value.

Arithmetic: Machine Determinism

$\text{det} : (S_0 \rightarrow S_1) \rightarrow (S_0 \rightarrow S_2) \rightarrow S_1 \equiv S_2$

- > At every evaluation step, there is exactly one transition the machine can take.
- > Proof by case distinction on both transitions.

Arithmetic: Machine Confluence

stuck : $(\vdash [] \downarrow v) \rightarrow s \rightarrow \perp$

confluence : $(s \rightarrow^* s_1) \rightarrow (s \rightarrow^* s_2)$
 $\rightarrow (\forall \{s'\} \rightarrow s_1 \rightarrow s' \rightarrow \perp) \rightarrow (\forall \{s'\} \rightarrow s_2 \rightarrow s' \rightarrow \perp)$
 $\rightarrow s_1 \equiv s_2$

- > A lifted version of **det**.
- > Proof by induction on the two traces:
 - > Both **base**: trivial.
 - > Both **step**: apply **det** on the single step; recursive call on the tails.
 - > One **step**, one **base**: the stuck hypothesis contradicts the assumption that s_1 and s_2 are final states.

Arithmetic: Completeness

`complete` : $e \Downarrow v \rightarrow (\vdash \text{stack} \uparrow e) \rightarrow^* (\vdash \text{stack} \downarrow v)$

- > Evaluating an expression e in a set context, returns v .
- > Proof by induction on the big-step derivation $e \Downarrow v$.
- > Each big-step constructor corresponds to a sequence of machine transitions.

Arithmetic: Completeness

complete (if e then e₁ else e₂) v stack (if-then p p₁) =
(stack ↑ e₀ ⊞ e₁)

→ { step-⊞ ₀ }	stack :: ○ ⊞ ₀ e ₁ ↑ e ₀
→ { complete (stack :: ○ ⊞ ₀ e ₁) .. }	stack :: ○ ⊞ ₀ e ₁ ↓ _
→ { step-⊞ ₁ }	stack :: _ ⊞ ₁ ○ ↑ e ₁
→ { complete (stack :: _ ⊞ ₁ ○) e ₁ - p ₁ }	stack :: _ ⊞ ₁ ○ ↓ _
→ { step-⊞ ₂ }	stack ↓ v

Arithmetic: Correctness

correct : $(\vdash [] \uparrow e) \rightarrow^* (\vdash [] \downarrow v) \rightarrow e \Downarrow v$

- > By totality, obtain some v' with $e \Downarrow v'$.
- > By completeness, build a canonical run $\vdash [] \uparrow e \rightarrow^* \vdash [] \downarrow v'$.
- > Confluence of the canonical and given run yields $v' \equiv v$.
- > Transport $e \Downarrow v'$ along the equality to obtain $e \Downarrow v$.

Arithmetic: Small-Step Semantics

data $_ \rightarrow _ : \vdash T \rightarrow \vdash T \rightarrow \text{Set}$ **where**

$\xi\text{-}\boxplus_1$: $e_1 \rightarrow e_1' \rightarrow (e_1 \boxplus e_2) \rightarrow (e_1' \boxplus e_2)$

$\xi\text{-}\boxplus_2$: $e_2 \rightarrow e_2' \rightarrow (\backslash n_1 \boxplus e_2) \rightarrow (\backslash n_1 \boxplus e_2')$

$\beta\text{-}\boxplus$: $(\backslash n_1 \boxplus \backslash n_2) \rightarrow \backslash (n_1 + n_2)$

$\xi\text{-if}$: $e \rightarrow e' \rightarrow (\text{if } e \text{ then } e_1 \text{ else } e_2) \rightarrow (\text{if } e' \text{ then } e_1 \text{ else } e_2)$

$\beta\text{-if-then}$: $(\text{if } \backslash \text{true} \text{ then } e_1 \text{ else } e_2) \rightarrow e_1$

$\beta\text{-if-else}$: $(\text{if } \backslash \text{false} \text{ then } e_1 \text{ else } e_2) \rightarrow e_2$

...

- > ξ -rules: reduce a sub-expression in context.
- > β -rules: perform a single computation step.
- > Reflexive-transitive closure $\rightarrow^*$.

Arithmetic: Big-Step $\leftrightarrow$ Small-Step

$\text{simulate} : e \Downarrow v \rightarrow e \rightarrow^* (\backslash v)$
 $\text{simulate}^R : e \rightarrow^* (\backslash v) \rightarrow e \Downarrow v$

> **simulate**: induction on the derivation; apply lifted ξ -rules, and then the appropriate β -rule.

> **simulate**^R: via totality + **simulate** + confluence (values $\backslash v$ are stuck).

$\rightarrow_{\boxplus}^* : \forall \{e_1 \ e_1' \ e_2 : \vdash \text{Nat}\} \rightarrow e_1 \rightarrow^* e_1' \rightarrow (e_1 \boxplus e_2) \rightarrow^* (e_1' \boxplus e_2)$

$\rightarrow_{\boxplus}^* \text{base} = \text{base}$

$\rightarrow_{\boxplus}^* (\text{step } r \ p) = \text{step } (\xi_{\boxplus} \ r) (\rightarrow_{\boxplus}^* p)$

$\rightarrow_{\boxplus}^* : \forall \{n : \mathbb{N}\} \{e_2 \ e_2' : \vdash \text{Nat}\} \rightarrow e_2 \rightarrow^* e_2' \rightarrow (\backslash n \boxplus e_2) \rightarrow^* (\backslash n \boxplus e_2')$

Arithmetic: Denotational Semantics

$$\begin{aligned} \llbracket _ \rrbracket &: \vdash T \rightarrow \models T \\ \llbracket v \rrbracket &= v \\ \llbracket e_1 \boxplus e_2 \rrbracket &= \llbracket e_1 \rrbracket + \llbracket e_2 \rrbracket \\ \llbracket \text{if } e \text{ then } e_1 \text{ else } e_2 \rrbracket &\text{ with } \llbracket e \rrbracket \\ \dots \mid \text{true} &= \llbracket e_1 \rrbracket \\ \dots \mid \text{false} &= \llbracket e_2 \rrbracket \end{aligned}$$
$$\begin{aligned} \text{simulate} &: e \Downarrow v \rightarrow \llbracket e \rrbracket \equiv v \\ \text{simulate}^R &: \llbracket e \rrbracket \equiv v \rightarrow e \Downarrow v \end{aligned}$$

- > $\llbracket _ \rrbracket$ evaluates expressions recursively.
- > simulate : by induction on the derivation tree.
- > simulate^R : by induction on the expression.

Outline

1. Introduction
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work
7. Conclusion

Exceptions: Syntax

```
data Exception : Set where
  err : Exception
```

```
data  $\vdash$ _ : Type  $\rightarrow$  Set where
  ...
  throw      : Exception  $\rightarrow \vdash$  T
  try_catch_ :  $\vdash$  T  $\rightarrow$  (Exception  $\rightarrow \vdash$  T)  $\rightarrow \vdash$  T
```

- > Evaluation can now produce a value *or* raise an exception.
- > `throw` raises an exception; `try-catch` evaluates the body and handles an exception if thrown.

Exceptions: Big-Step Semantics

```
data Result (T : Type) : Set where
  return :  $\models T \rightarrow$  Result T
  raise   : Exception  $\rightarrow$  Result T
```

$e \Downarrow v = e \Downarrow^x \text{return } v$
 $e \Uparrow \text{ex} = e \Downarrow^x \text{raise ex}$

```
data _ $\Downarrow^x$ _ :  $\vdash T \rightarrow$  Result T  $\rightarrow$  Set where
  _ $\boxplus$ _      :  $e_1 \Downarrow n_1 \rightarrow e_2 \Downarrow n_2 \rightarrow (e_1 \boxplus e_2) \Downarrow (n_1 + n_2)$ 
   $\boxplus$ -exn1 :  $e_1 \Uparrow \text{ex} \rightarrow (e_1 \boxplus e_2) \Uparrow \text{ex}$ 
   $\boxplus$ -exn2 :  $e_1 \Downarrow n_1 \rightarrow e_2 \Uparrow \text{ex} \rightarrow (e_1 \boxplus e_2) \Uparrow \text{ex}$ 
  ...
  throw-rule  : throw ex  $\Uparrow$  ex
  try-return  :  $e \Downarrow v \rightarrow (\text{try } e \text{ catch } h) \Downarrow v$ 
  try-catch   :  $e \Uparrow \text{ex} \rightarrow h \text{ ex } \Downarrow^x r \rightarrow (\text{try } e \text{ catch } h) \Downarrow^x r$ 
```

> Each constructor gains additional exception-propagation rules (e.g. $\boxplus$ -exn₁).

Exceptions: Totality

$\text{total} : \forall (e : \vdash T) \rightarrow \exists (\lambda r \rightarrow e \Downarrow^r r)$

- > Follows the same structure as arithmetic: structural induction on e .
- > The result is now a `Result T`: either `return v` or `raise ex`.

$\text{total } (\text{throw } ex) = \text{raise } ex$, throw-rule

$\text{total } (\text{try } e \text{ catch } h) \text{ with } \text{total } e$

... | `return v` , $p = \text{return } v$, try-return p

... | `raise ex` , $p = \text{let } (r , q) = \text{total } (h \text{ ex}) \text{ in } r$, try-catch $p \ q$

Exceptions: Abstract Machine

data Frame : Type $\rightarrow$ Type $\rightarrow$ Set **where**

$\dots$
catch : (Exception $\rightarrow \vdash T$) $\rightarrow$ Frame T T

- > **catch** h: stores the exception handler.
- > If the body raises an exception, frames should be discarded until the nearest handler is found.

Exceptions: Abstract Machine

data State (Answer : Type) : **Set** **where**

$\vdash_{\uparrow}$: Stack Answer T $\rightarrow \vdash$ T $\rightarrow$ State Answer

$\vdash_{\downarrow}$: Stack Answer T $\rightarrow \models$ T $\rightarrow$ State Answer

$\vdash_{\leftarrow}$: Stack Answer T $\rightarrow$ Exception $\rightarrow$ State Answer

step-throw : $\vdash$ stack $\uparrow$ throw ex $\rightarrow \vdash$ stack $\leftarrow$ ex

step-try : $\vdash$ stk $\uparrow$ (try e catch h) $\rightarrow \vdash$ stk :: catch h $\uparrow$ e

step-catch-ok : $\vdash$ stk :: catch h $\downarrow$ v $\rightarrow \vdash$ stk $\downarrow$ v

step-catch-exn : $\vdash$ stack :: catch h $\leftarrow$ ex $\rightarrow \vdash$ stack $\uparrow$ h ex

step-drop- $\boxplus_0$: $\vdash$ stack :: ($\circ \boxplus_0$ e₂) $\leftarrow$ ex $\rightarrow \vdash$ stack $\leftarrow$ ex

- > Unwind mode propagates the exception, and scans the stack for the nearest catch frame
- > Final states: $\vdash [] \downarrow v$ (return) and $\vdash [] \leftarrow$ ex (unhandled exception).

Exceptions: Abstract Machine

- > Why not use a generic `step-drop` rule that discards any frame?

`step-drop` : $\forall \{f : \text{Frame } T_1 \ T_2\}$
 $\vdash \text{stack} :: f < \text{ex} \quad \rightarrow \vdash \text{stack} < \text{ex}$

`step-catch-exn` : $\vdash \text{stack} :: \text{catch } h < \text{ex} \quad \rightarrow \vdash \text{stack} \uparrow h \text{ ex}$

- > Causes the abstract machine to be non-deterministic.

Exceptions: Machine Determinism

$\text{det} : (S_0 \rightarrow S_1) \rightarrow (S_0 \rightarrow S_2) \rightarrow S_1 \equiv S_2$

- > At every evaluation step, there is exactly one transition the machine can take.
- > Proof by case distinction on both transitions.

Exceptions: Machine Confluence

$\text{stuck-}\downarrow : (\vdash [] \downarrow v) \rightarrow s \rightarrow \perp$
 $\text{stuck-}\prec : (\vdash [] \prec \text{ex}) \rightarrow s \rightarrow \perp$
 $\downarrow \neq \prec : (\vdash [] \downarrow v) \equiv (\vdash [] \prec \text{ex}) \rightarrow \perp$

$\text{confluence} : (s \rightarrow^* s_1) \rightarrow (s \rightarrow^* s_2)$
 $\rightarrow (\forall \{s'\} \rightarrow s_1 \rightarrow s' \rightarrow \perp) \rightarrow (\forall \{s'\} \rightarrow s_2 \rightarrow s' \rightarrow \perp)$
 $\rightarrow s_1 \equiv s_2$

- > A lifted version of `det`: same three-case induction as Arithmetic.
- > There are now *two* final states.

Exceptions: Completeness

`complete` : $e \Downarrow v \rightarrow (\vdash \text{stk} \uparrow e) \rightarrow^* (\vdash \text{stk} \downarrow v)$
`complete-exn` : $e \Uparrow \text{ex} \rightarrow (\vdash \text{stk} \uparrow e) \rightarrow^* (\vdash \text{stk} < \text{ex})$

- > Two mutually recursive lemmas: one for each final state.
- > `try-return`: body succeeds; `complete` evaluates it, `step-catch-ok` discards the handler frame.
- > `try-catch`: `complete-exn` drives the body to `<`; `step-catch-exn` activates the handler; `complete` finishes.

Exceptions: Completeness

```
complete (try e catch h) v stk (try-catch {ex = ex} pe ph) =  
  step step-try  
    (→*-transitive (complete-exn e ex (stk :: catch h) pe)  
      (step step-catch-exn  
        (complete (h ex) v stk ph))))
```

- > Push catch h frame, transition to ↑ mode.
- > complete-exn drives the body to < at the handler frame.
- > step-catch-exn activates the handler; complete evaluates it to ↓.

Exceptions: Correctness

`correct` : $(\vdash [] \uparrow e) \rightarrow^* (\vdash [] \downarrow v) \rightarrow e \Downarrow v$
`correct-exn` : $(\vdash [] \uparrow e) \rightarrow^* (\vdash [] \prec ex) \rightarrow e \Uparrow ex$

`correct e v tr` **with** `total e`

`... | return v' , p = .. -- complete  $\rightarrow$  confluence  $\rightarrow v' \equiv v$`
`... | raise ex , p = .. -- complete-exn  $\rightarrow$  confluence  $\rightarrow \downarrow \not\equiv \prec \rightarrow ex\text{-}false$`

- > Totality splits into two cases.
 - > Both are $\downarrow$ -runs; confluence yields $v' \equiv v$.
 - > Both are $\prec$ -runs; confluence yields $ex' \equiv ex$.
 - > One is an $\prec$ -run, the other is a $\downarrow$ -run; confluence yields $\downarrow \equiv \prec$, contradiction.

Exceptions: Small-Step Semantics

data $_ \rightarrow _$: $\vdash T \rightarrow \vdash T \rightarrow \text{Set}$ **where**

...

$\beta\text{-}\boxplus\text{-exn}_1$: $(\text{throw ex} \boxplus e_2) \rightarrow \text{throw ex}$

$\beta\text{-}\boxplus\text{-exn}_2$: $(\text{' } n_1 \boxplus \text{throw ex}) \rightarrow \text{throw ex}$

$\xi\text{-try}$: $e \rightarrow e' \rightarrow (\text{try } e \text{ catch } h) \rightarrow (\text{try } e' \text{ catch } h)$

$\beta\text{-try-ok}$: $(\text{try } (\text{' } v) \text{ catch } h) \rightarrow \text{' } v$

$\beta\text{-try-throw}$: $(\text{try } (\text{throw ex}) \text{ catch } h) \rightarrow h \text{ ex}$

- > $\beta\text{-exn}$ rules propagate **throw** upward through each constructor.
- > $\beta\text{-try-throw}$ is the only rule that *consumes* a **throw**, activating the handler.

Exceptions: Big-Step $\leftrightarrow$ Small-Step

`resultExpr` : $\text{Result } T \rightarrow \vdash T$
`resultExpr` (return v) = $\backslash v$
`resultExpr` (raise ex) = throw ex

`simulate` : $e \Downarrow^r r \rightarrow e \rightarrow^* \text{resultExpr } r$
`simulateR` : $e \rightarrow^* (\text{resultExpr } r) \rightarrow e \Downarrow^r r$

- > `resultExpr` maps a result to its stuck expression ($\backslash v$ or throw ex).
- > **simulate**: induction on the derivation; exception cases use β -exn rules to bubble throw up.
- > **simulate^R**: totality + `simulate` + confluence + `resultExpr-inj` then transport.

Exceptions: Denotational Semantics

```
interpret :  $\vdash T \rightarrow \text{Result } T$   
interpret (throw ex)      = raise ex  
interpret (try e catch h) with interpret e  
... | raise ex = interpret (h ex)  
... | return v = return v  
...
```

```
simulate :  $e \Downarrow^r r \rightarrow \text{interpret } e \equiv r$   
simulateR :  $\text{interpret } e \equiv r \rightarrow e \Downarrow^r r$ 
```

- > `interpret` propagates exceptions structurally, mirroring big-step.
- > `simulate`: by induction; `rewrite` collapses the equality at each constructor.
- > `simulateR`: reads the result from the equality and reconstructs the derivation.

Outline

1. Introduction
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work
7. Conclusion

STLC: Contexts and Environments

- > Contexts record the types of bound variables.
- > Environments record the values of bound variables of a context.

data Context : Set where

$\emptyset$: Context

$-, -$: Context $\rightarrow$ Type $\rightarrow$ Context

data Environment : Context $\rightarrow$ Set where

$\emptyset$: Environment $\emptyset$

$-, -$: Environment $\Gamma \rightarrow \models T \rightarrow$ Environment (Γ, T)

STLC: De Bruijn Indices

```
data _∈_ : Type → Context → Set where
  Z      : T ∈ (Γ , T)
  S_     : T1 ∈ Γ → T1 ∈ (Γ , T2)

lookupe : Environment Γ → T ∈ Γ → Value T
lookupe (γ , x) Z = x
lookupe (γ , x) (S y) = lookupe γ y
```

STLC: Syntax and Intrinsic Typing

```
data Type : Set where
  Bool : Type; Nat : Type;  $\_ \Rightarrow \_$  : Type  $\rightarrow$  Type  $\rightarrow$  Type

data  $\_ \vdash \_$  ( $\Gamma$  : Context) : Type  $\rightarrow$  Set where
  Nat :  $\mathbb{N} \rightarrow \Gamma \vdash \text{Nat}$ ; Bool :  $\mathbb{B} \rightarrow \Gamma \vdash \text{Bool}$ 
  Var :  $T \in \Gamma \rightarrow \Gamma \vdash T$ 
   $\lambda \_$  :  $(\Gamma, T_1) \vdash T_2 \rightarrow \Gamma \vdash (T_1 \Rightarrow T_2)$ 
   $\_ \cdot \_$  :  $\Gamma \vdash (T_1 \Rightarrow T_2) \rightarrow \Gamma \vdash T_1 \rightarrow \Gamma \vdash T_2$ 
```

STLC: Big-Step Semantics

```
record Closure (A B : Type) : Set where
  field {Γc} : Context; δc : Environment Γc; ec : (Γc , A) ⊢ B
```

$\models (T_1 \Rightarrow T_2) = \text{Closure } T_1 \ T_2$

```
data _⊢_↓_ : Environment Γ → Γ ⊢ T → ⊢ T → Set where
  VAR : δ ⊢ Var x ↓ lookupe δ x
  ABS : δ ⊢ λ e ↓ ⟨ δ , e ⟩
  app : δ ⊢ e0 ↓ ⟨ δc , ec ⟩ → δ ⊢ e1 ↓ v1 → (δc , v1) ⊢ ec ↓ v
        → δ ⊢ e0 · e1 ↓ v
```

- > Evaluation is *environment-based*: variables resolved by $\text{lookup}_e \ \delta$.
- > A λ captures the current environment δ as a closure.
- > Application evaluates function, argument, then runs the closure body in δ^c , v_1 .

STLC: Determinism

deterministic : $\delta \vdash e \Downarrow v_1 \rightarrow \delta \vdash e \Downarrow v_2 \rightarrow v_1 \equiv v_2$

- > Proof by simultaneous induction on both derivations.
- > NAT, BOOL, VAR, ABS: trivial.
- > app: induction applied three times, on function value, argument value, and closure body.

STLC: Totality

```
{-# TERMINATING #-}  
total :  $\forall (e : \Gamma \vdash T) \rightarrow \exists (\lambda v \rightarrow \delta \vdash e \Downarrow v)$   
total (e1 · e2) with total e1 | total e2  
... |  $\langle \delta^c, e^c \rangle ' p \mid v_2 ' q$  with total { $\delta = \delta^c, v_2$ } ec  
... |  $v ' r = v ' \text{app } p \ q \ r$ 
```

- STLC is strongly normalising (always terminates with a value), but Agda cannot verify this structurally.
- The `{-# TERMINATING #-}` pragma can be used to disable the check, leaving the burden of proof to the programmer.

STLC: Abstract Machine

data Frame : Type $\rightarrow$ Type $\rightarrow$ Set **where**

...

app₁ : Hole $\rightarrow$ $\Gamma \vdash A$ $\rightarrow$ Frame B (A $\Rightarrow$ B)

app₂ : $\models (A \Rightarrow B) \rightarrow$ Hole $\rightarrow$ Frame B A

app₃ : Environment $\Gamma \rightarrow \models (A \Rightarrow B) \rightarrow \models A \rightarrow$ Hole $\rightarrow$ Frame B B

- > Application decomposes into *three* sequential frames.
- > app₃ stores the *caller's environment* δ so it can be restored after the call.
- > mirrors the behaviour in the big-step app rule.

STLC: Abstract Machine

data State (Answer : Type) : **Set** **where**

$_ \vdash _ \uparrow _$: Environment $\Gamma \rightarrow$ Stack Answer $A \rightarrow \Gamma \vdash A \rightarrow$ State Answer

$_ \vdash _ \downarrow _$: Environment $\Gamma \rightarrow$ Stack Answer $A \rightarrow \models A \rightarrow$ State Answer

step- λ : $\delta \vdash \text{stack} \uparrow \lambda e \rightarrow \delta \vdash \text{stack} \downarrow \langle \delta, e \rangle$

step- $\cdot_2$: $\delta \vdash \text{stack} :: \text{app}_2 \langle \delta^c, e^c \rangle \circ \downarrow v_2$
 $\rightarrow (\delta^c, v_2) \vdash \text{stack} :: \text{app}_3 \delta \langle \delta^c, e^c \rangle v_2 \circ \uparrow e^c$

step- $\cdot_3$: $\delta \vdash \text{stack} :: \text{app}_3 \delta' \text{clo } v_2 \circ \downarrow v_3 \rightarrow \delta' \vdash \text{stack} \downarrow v_3$

- > Each state carries the *current environment* δ alongside the stack.
- > **step- $\cdot_2$** : switches to closure environment δ^c , v_2 to evaluate the body.
- > **step- $\cdot_3$** : returns to the caller's environment δ' from the app_3 frame.

STLC: Completeness

complete : $\delta \vdash e \Downarrow v \rightarrow (\delta \vdash \text{stack} \uparrow e) \rightarrow^* (\delta \vdash \text{stack} \downarrow v)$

- > Proof by induction on the big-step derivation; generalised over an arbitrary stack.
- > app case: three recursive calls to handle $\text{step-}\cdot$, $\text{step-}\cdot_1$, $\text{step-}\cdot_2$, $\text{step-}\cdot_3$.

STLC: Completeness

$$\begin{array}{l}
 \text{complete } \delta \text{ stack } (e_0 \cdot e_1) \ v \ (\text{app } \{\delta^c = \delta^c\} \ \{e^c\} \ \{v_1\} \ p \ p_1 \ p_2) = \\
 \text{proof} \quad \delta \vdash \text{stack } \uparrow (e_0 \cdot e_1) \\
 \rightarrow \langle \text{step-}\cdot \rangle \quad \delta \vdash \text{stack} :: \text{app}_1 \circ e_1 \uparrow e_0 \\
 \rightarrow \langle \text{complete } \delta \ (\text{stack} :: \text{app}_1 \circ e_1) \ e_0 \ \langle \delta^c, e^c \rangle \ p \rangle \dots \\
 \rightarrow \langle \text{step-}\cdot_1 \rangle \dots \rightarrow \langle \text{complete } \delta \ (\text{stack} :: \text{app}_2 \circ) \ e_1 \ v_1 \ p_1 \rangle \dots \\
 \rightarrow \langle \text{step-}\cdot_2 \rangle \\
 \quad (\delta^c, v_1) \vdash \text{stack} :: \text{app}_3 \ \delta \ \langle \delta^c, e^c \rangle \ v_1 \circ \uparrow e^c \\
 \rightarrow \langle \text{complete } (\delta^c, v_1) \ (\text{stack} :: \text{app}_3 \ \delta \circ v_1) \ e^c \ v \ p_2 \rangle \dots \\
 \rightarrow \langle \text{step-}\cdot_3 \rangle \quad \delta \vdash \text{stack } \downarrow v \quad \blacksquare
 \end{array}$$

- > Six transitions; three IHs handle the sub-evaluations.
- > Environment switches at $\text{step-}\cdot_2$ (enter closure) and back at $\text{step-}\cdot_3$ (exit).

STLC: Correctness

correct : $(\delta \vdash [] \uparrow e) \rightarrow^* (\delta \vdash [] \downarrow v) \rightarrow \delta \vdash e \Downarrow v$

- > Follows the same confluence argument.
- > Obtain v' , $p : e \Downarrow v$ from **total** e ; build run via **complete**.
- > Confluence of both runs run yields $\delta \vdash [] \downarrow v' \equiv \delta \vdash [] \downarrow v$.
- > Injectively, $v' \equiv v$; therefore we can obtain $e \Downarrow v$.

STLC: Small-Step Semantics

```
data Normal :  $\Gamma \vdash T \rightarrow$  Set where
  nat : Normal (Nat n);  bool : Normal (Bool b);  closure : Normal ( $\lambda e$ 

data  $\rightarrow$  :  $\Gamma \vdash T \rightarrow \Gamma \vdash T \rightarrow$  Set where
   $\beta$ - $\boxplus$       : Nat n1  $\boxplus$  Nat n2  $\rightarrow$  Nat (n1 + n2)
   $\beta$ -app       :  $\lambda e \cdot v \rightarrow e [ v ]$ 
   $\xi$ -app-left  :  $e_1 \rightarrow e_1' \rightarrow e_1 \cdot e_2 \rightarrow e_1' \cdot e_2$ 
   $\xi$ -app-right : Normal e1  $\rightarrow e_2 \rightarrow e_2' \rightarrow e_1 \cdot e_2 \rightarrow e_1 \cdot e_2'$ 
  ...
```

- > **Small-step is Substitution-based:** β -app uses $[_]$, substituting the argument into the body (shifting indices in the process).
- > **Big-step is environment-based:** the rule app extends δ with the argument v .

STLC: Denotational Semantics

`evaluate` : Environment $\Gamma \rightarrow \Gamma \vdash T \rightarrow \models T$
`evaluate` δ (λe) = $\langle \delta, e \rangle$
`evaluate` δ $(e_1 \cdot e_2)$ **with** `evaluate` δ e_1
... | $\langle \delta^c, e^c \rangle$ = `apply` $\langle \delta^c, e^c \rangle$ (`evaluate` δ e_2)

`apply` : $\models (A \Rightarrow B) \rightarrow \models A \rightarrow \models B$
`apply` $\langle \delta^c, e^c \rangle$ v = `evaluate` (δ^c, v) e^c

{-# TERMINATING #-}

`simulate` : `evaluate` δ $e \equiv v \rightarrow \delta \vdash e \Downarrow v$

`simulate`^R : $\delta \vdash e \Downarrow v \rightarrow \text{evaluate } \delta \ e \equiv v$

- > Same TERMINATING caveat: the closure body call is not structurally smaller.
- > `simulate/simulate`^R: each direction by induction; `rewrite` at each constructor.

STLC: Big-Step $\leftrightarrow$ Small-Step

$$\begin{aligned} & e \ [\ v_1 \] \mapsto^* \text{embed } v_2 \\ \rightarrow & \delta \vdash (\lambda e) \cdot v_1 \Downarrow v_2 \end{aligned}$$

> The `simulateR` direction for `app` must convert a small-step trace into a big-step derivation – but big-step is environment-based, so the result value v_2 must already be known.

> Alternatively, if can quote values back into syntax:

$$\begin{aligned} & \delta \vdash s \ [\ \backslash \ v \] \Downarrow v_2 \\ \rightarrow & \delta \vdash s \ [\ \backslash \ \langle \delta^c, e^c \rangle \cdot \backslash \ v_1 \] \Downarrow v_2 \end{aligned}$$

Outline

1. Introduction
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
- 6. Related Work**
7. Conclusion

Related Work

- Hutton derives machines by CPS transformation and defunctionalization.
- This thesis derives machines from big-step semantics by reifying frames.
- Conceptually: $\text{frame} \approx \text{defunctionalized continuation}$.
- Hutton and Wright also study exceptions and verified compilation.

Outline

1. Introduction
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work
7. Conclusion

Conclusion: Summary

- > Presented a systematic process for deriving an abstract machine from big-step semantics for Arithmetic, Exceptions, and STLC.
- > The derivation proceeds by:
 - > representing the process of evaluation explicitly via **frames**.
 - > defining machine states as a **stack of frames** in three modes: *eval*, *return*, and (for exceptions) *unwind*;
 - > defining **transition rules** by mimicking the in-order evaluation of terms.
- > **Completeness**: the machine trace produces the same value as a big-step derivation.
- > **Correctness**: totality yields a run of the machine to v' ; any run to v must converge to the same result.
- > **Compositionality**: derive an equivalence between other forms of semantics using big-step as an intermediary.

Conclusion: Limitations and Future Work

- > Current route assumes totality and does not account for non-terminating programs.
- > STLC totality currently uses a termination pragma.
- > STLC small-step $\leftrightarrow$ big-step equivalence is open.

Appendix: Propositions as Types

> Propositions are types, proofs are terms.

```
data  $\top$  : Set where  
  tt :  $\top$ 
```

```
data  $\perp$  : Set where
```

```
ex-falso : {P : Set}  $\rightarrow \perp \rightarrow P$   
ex-falso ()
```

Appendix: Connectives in Agda

$\neg$: Set $\rightarrow$ Set

$\neg P = P \rightarrow \perp$

data $_ \times _$ (P Q : Set) : Set where

$_, _$: P $\rightarrow$ Q $\rightarrow$ P $\times$ Q

data Σ (A : Set) (B : A $\rightarrow$ Set) : Set where

$_, _$: (x : A) $\rightarrow$ B x $\rightarrow$ Σ A B

data $_ \sqcup _$ (P Q : Set) : Set where

inl : P $\rightarrow$ P $\sqcup$ Q

inr : Q $\rightarrow$ P $\sqcup$ Q

Appendix: Equality and Naturals

```
data _≡_ {A : Set} (x : A) : A → Set where  
  refl : x ≡ x
```

```
data IN : Set where  
  zero : IN  
  suc  : IN → IN
```

```
_+_ : IN → IN → IN  
zero + m = m  
suc n + m = suc (n + m)
```

Appendix: Rewrite and Induction

```
n+0≡n : (n : ℕ) → n + zero ≡ n
n+0≡n zero = refl
n+0≡n (suc n) rewrite n+0≡n n = refl
```

```
n+sm≡n : (n m : ℕ) → n + suc m ≡ suc (n + m)
n+sm≡n zero m = refl
n+sm≡n (suc n) m rewrite n+sm≡n n m = refl
```

```
+commutative : (n m : ℕ) → n + m ≡ m + n
+commutative zero m rewrite n+0≡n m = refl
+commutative (suc n) m rewrite +commutative n m = sym (n+sm≡n m n)
```

Appendix: Termination

```
diverging :  $\perp$   
diverging = diverging
```

- > Non-terminating definitions would cause the logic to be inconsistent.
- > Agda's termination checker requires every recursive call to be on a structurally smaller argument.
- > A `{-# TERMINATING #-}` pragma leaves the burden to the programmer.